



Tec Gurus
SOMOS Y FORMAMOS EXPERTOS EN T.I.

CURSO

Linux Desde Cero

CLASE 1

Fundamentos, archivos y permisos

www.tecgurus.net

INSTRUCTOR

Ing. Abimael Domínguez



AGENDA • 09:00–14:00

Clase 1 — Fundamentos, archivos y permisos

Archivos: capítulos 1, 3 y 7.1–7.15 de 7.

HORA	ACTIVIDAD
09:00–09:20	Verificar el entorno con <code>whoami</code> , <code>hostname</code> , <code>pwd</code> y <code>/etc/os-release</code> .
09:20–10:00	Kernel, distribución, shell, terminal y escritorio.
10:00–10:40	Usuarios, grupos y <code>sudo</code> ; práctica controlada.
10:40–11:00	Flujo mínimo de <code>apt</code> .
11:00–11:30	Receso.
11:30–12:15	Jerarquía, rutas y tipos de archivo.
12:15–13:05	<code>pwd</code> , <code>ls</code> , <code>cd</code> , <code>mkdir</code> , <code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> y <code>file</code> .
13:05–13:40	Enlaces, permisos, propietario y grupo.
13:40–13:55	Reto 3: directorio compartido.
13:55–14:00	Evidencia y cierre.

No recortar: navegación, operaciones y permisos.

Recortar primero: comparación de distribuciones y detalles de cuentas.

Índice interactivo

Selecciona un apartado para ir directamente a su contenido. En cada encabezado encontrarás el enlace para volver aquí.

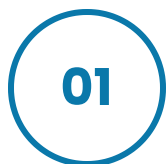
CAPÍTULO 1	
Introducción a Linux	
Objetivos	→
Antes de empezar	→
1.1 ¿Qué es Linux?	→
• Comprobar el sistema	→
1.2 ¿Qué son las distribuciones?	→
• Administración esencial de paquetes	→
• Ayuda antes de ejecutar	→
1.3 Entorno de trabajo: shell y entorno gráfico	→
1.4 Usuarios y grupos	→
• ¿Qué hace getent?	→
• Crear un usuario de laboratorio	→
• Sintaxis general	→
• Ejemplo resuelto para copiar	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 1 — Inventario reproducible	→
• Criterios de comprobación	→
Checklist	→

Índice interactivo

Sistema de archivos y comandos esenciales del shell.

CAPÍTULO 3	
Estructura del sistema de archivos	
Objetivos	→
Antes de empezar	→
3.1 Tipos de archivo	→
3.2 Enlaces	→
3.3 Rutas o paths	→
3.4 Jerarquía principal	→
3.5 Acceso a sistemas de archivos	→
3.6 Permisos	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 3 — Directorio compartido	→
· Criterios de comprobación	→
Checklist	→

CAPÍTULO 7.1–7.15	
El shell	
Objetivos	→
Antes de empezar	→
7.1–7.3 Shell, comandos y directorio personal	→
7.4 Listar: ls	→
7.5–7.8 Directorios y navegación	→
7.9 Unidades y montajes	→
7.10 Copiar: cp	→
7.11 Mover y renombrar: mv	→
7.12 Enlaces: ln	→
7.13 Borrar: rm	→
7.14 Identificar: file y stat	→
7.15 Permisos y propiedad	→



FUNDAMENTOS

Introducción a Linux

Kernel, distribución, terminal, shell, paquetes, usuarios, grupos y privilegios administrativos.

Capítulo 1

Ubuntu 24.04

apt

sudo

Objetivos

[Índice ↑](#)

- Distinguir kernel, distribución, shell, terminal y escritorio.
- Identificar la distribución y el kernel de una máquina.
- Consultar ayuda e instalar paquetes en Ubuntu.
- Trabajar con usuarios, grupos y privilegios mediante `sudo`.

Antes de empezar

[Índice ↑](#)

Abre una terminal y sitúate en la raíz del curso:

```
cd ~/linux-desde-cero
test -f README.md && echo "Directorio correcto"
```

TERMINAL · BASH

Si el mensaje no aparece, revisa dónde clonaste el repositorio antes de continuar. En este capítulo instalarás o crearás temporalmente:

- el paquete `tree`, para visualizar directorios;
- un grupo llamado `devops-lab`;
- un usuario de práctica llamado `alumno`;
- la carpeta `~/curso-linux/evidencias` para guardar resultados.

1.1 ¿Qué es Linux?

[Índice ↑](#)

Linux es el **kernel**: administra CPU, memoria, dispositivos, procesos y sistemas de archivos. Una distribución combina ese kernel con herramientas, bibliotecas, un gestor de paquetes y decisiones de configuración.



SALIDA REPRESENTATIVA

hardware → kernel Linux → herramientas del sistema → aplicaciones → usuario

Comprobar el sistema

[Índice ↑](#)

TERMINAL · BASH

```
uname -r  
cat /etc/os-release
```

SALIDA REPRESENTATIVA



SALIDA REPRESENTATIVA

```
6.8.0-xx-generic  
PRETTY_NAME="Ubuntu 24.04.x LTS"  
ID=ubuntu  
VERSION_ID="24.04"
```

- `uname -r`: muestra la versión del kernel en ejecución.
- `/etc/os-release`: describe la distribución, no el kernel.
- `-r`: solicita el *kernel release*.

1.2 ¿Qué son las distribuciones?

[Índice ↑](#)

Las distribuciones comparten el kernel, pero cambian herramientas, versiones, soporte y paquetes.

Familia	Ejemplos	Gestor habitual
Debian	Ubuntu, Debian	apt / dpkg
RHEL	Red Hat Enterprise Linux, Fedora, Rocky Linux	dnf / rpm
Arch	Arch Linux, Manjaro	pacman

El laboratorio usa Ubuntu. Aprender los conceptos y consultar ayuda es más importante que memorizar tres gestores.

Administración esencial de paquetes

[Índice ↑](#)

TERMINAL · BASH

```
sudo apt update
apt search tree
apt show tree
sudo apt install tree
tree --version
```

- `apt update` : actualiza el índice local; no actualiza todavía los programas.
- `search` : busca por nombre y descripción.
- `show` : enseña versión, tamaño y dependencias.
- `install` : instala el paquete solicitado.
- `sudo` : ejecuta sólo ese comando con privilegios administrativos.

En este capítulo **no eliminamos `tree` todavía**, porque se utiliza en la práctica final. Cuando termines y sólo si deseas retirarlo, la sintaxis es:



TERMINAL · BASH

```
sudo apt remove tree
```

En una distribución con DNF, el flujo equivalente usa `dnf search`, `dnf info`, `dnf install` y `dnf remove`.

Ayuda antes de ejecutar

[Índice ↑](#)

TERMINAL · BASH

```
man uname
uname --help
type uname
command -v uname
```

- `man` : manual completo; sal con `q`.
- `--help` : resumen rápido de opciones.
- `type` : indica si el nombre es alias, builtin o ejecutable.
- `command -v` : muestra cómo lo resolverá el shell.

1.3 Entorno de trabajo: shell y entorno gráfico

[Índice ↑](#)

- **Terminal:** interfaz que recibe texto y muestra resultados.
- **Shell:** programa que interpreta comandos; Bash es el principal del curso.
- **Escritorio:** entorno gráfico como GNOME o KDE Plasma.
- **Servidor gráfico/compositor:** conecta aplicaciones gráficas, entrada y pantalla.

```
echo "$SHELL"
ps -p "$$" -o comm=
tty
```

SALIDA REPRESENTATIVA

```
/bin/bash
bash
/dev/pts/0
```

- `$SHELL` contiene el shell configurado para el usuario.
- `$$` es el PID del shell actual.
- `tty` identifica la terminal asociada a la sesión.

1.4 Usuarios y grupos

[Índice ↑](#)

Linux separa identidades y permisos mediante UID y GID.

```
whoami
id
groups
getent passwd "$USER"
```

SALIDA REPRESENTATIVA

```
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo)
```

- `uid`: identidad numérica del usuario.
- `gid`: grupo principal.
- `groups`: grupos adicionales; `sudo` suele conceder administración en Ubuntu.

¿Qué hace `getent`?

[Índice ↑](#)

`getent` significa **get entries**: obtener registros de las bases de datos que Linux tiene configuradas en `/etc/nsswitch.conf`.

Esto es importante porque los usuarios y grupos no siempre provienen únicamente de `/etc/passwd` y `/etc/group`. En una organización también pueden venir de LDAP, Active Directory u otro servicio de identidades. `getent` consulta esas fuentes mediante la configuración del sistema y presenta una salida uniforme.

SINTAXIS GENERAL

```
getent <base_de_datos> [clave]
```

EJEMPLOS RESUELTOS

```
getent passwd "$USER"
getent passwd root
getent group sudo
```

- `passwd`: base de datos de cuentas de usuario.

- `group` : base de datos de grupos.
- `"$USER"` : se sustituye automáticamente por tu usuario actual.
- `root` y `sudo` : claves concretas que queremos buscar.

Una salida de usuario tiene siete campos separados por :

```
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
```

└─ shell de inicio
├── directorio personal
│ └── descripción de la cuenta
│ ├── GID del grupo principal
│ │ └── UID del usuario
│ └── la contraseña/hash no se guarda aquí
└── nombre de usuario

La `x` no es la contraseña. Indica que la información protegida se guarda en `/etc/shadow`, que sólo puede leer `root`.

Para comprobar si una cuenta existe sin imprimir información innecesaria:

```
if getent passwd alumno > /dev/null; then
    echo "El usuario alumno existe"
else
    echo "El usuario alumno no existe"
fi
```

`getent` devuelve estado `0` cuando encuentra el registro y un estado distinto de cero cuando no lo encuentra. Por eso puede utilizarse directamente como condición de un `if`.

Crear un usuario de laboratorio

[Índice ↑](#)

Vamos a crear dos objetos diferentes:

Objeto	Nombre usado	Propósito
Grupo	devops-lab	Representa a un equipo que comparte permisos.
Usuario	alumno	Cuenta temporal con la que practicaremos administración.

El nombre del usuario es `alumno`; aparece como último argumento en comandos como `useradd`, `passwd`, `id` y `usermod`.

Sintaxis general

[Índice ↑](#)

```
sudo groupadd <nombre_grupo>
sudo useradd -U -m -s /bin/bash -G <nombre_grupo> <nombre_usuario>
sudo passwd <nombre_usuario>
sudo usermod -aG sudo <nombre_usuario>
id <nombre_usuario>
```

- `<nombre_grupo>`: grupo adicional al que pertenecerá la cuenta.
- `<nombre_usuario>`: nombre de la cuenta que quieres crear.
- Los marcadores `<...>` se sustituyen; no se escriben literalmente.

Ejemplo resuelto para copiar

[Índice ↑](#)

```
sudo groupadd devops-lab
sudo useradd -U -m -s /bin/bash -G devops-lab alumno
sudo passwd alumno
sudo usermod -aG sudo alumno
id alumno
```

Qué ocurre en cada línea:

1. `groupadd devops-lab` crea el grupo `devops-lab`.
 2. `useradd ... alumno` crea el usuario `alumno`.
 3. `passwd alumno` solicita dos veces una contraseña para `alumno`; mientras escribes no se muestran caracteres, pero el teclado sí está funcionando.
 4. `usermod -aG sudo alumno` agrega `alumno` al grupo administrativo `sudo`.
 5. `id alumno` confirma UID, grupo principal y grupos secundarios después del cambio.
- `-m`: crea el directorio personal.
 - `-U`: crea un grupo principal con el mismo nombre del usuario.
 - `-s`: define el shell de inicio.
 - `-G`: asigna grupos secundarios.
 - `-aG`: **añade** grupos; omitir `-a` puede reemplazar membresías existentes.

Comprueba que la cuenta y su home existen:

```
getent passwd alumno
id alumno
sudo ls -ld /home/alumno
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
alumno:x:1001:1001:./home/alumno:/bin/bash
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno),27(sudo),1002(devops-lab)
drwxr-x--- ... alumno alumno ... /home/alumno
```

Los números UID/GID pueden ser distintos. Lo importante es ver `alumno`, `/home/alumno`, `/bin/bash`, `sudo` y `devops-lab`. La nueva membresía de grupo se aplica al iniciar una sesión nueva; no necesitas cambiar de usuario para continuar el capítulo.

Limpieza, después de la práctica:

TERMINAL · BASH

```
sudo userdel -r alumno
sudo groupdel devops-lab
```

`userdel -r alumno` elimina la cuenta y `/home/alumno`. Ejecútalo únicamente cuando ya no necesites los archivos de ese usuario. Después se elimina el grupo temporal `devops-lab`.

Práctica guiada resuelta

[Índice ↑](#)

Objetivo: inspeccionar el host e instalar una herramienta.

Esta práctica se ejecuta con tu usuario habitual, no con la cuenta `alumno`. Creará `~/curso-linux/evidencias/sistema.txt` y después mostrará la estructura con `tree`.

```
mkdir -p ~/curso-linux/evidencias
{
  echo "Usuario: $(whoami)"
  echo "Kernel: $(uname -r)"
  grep '^PRETTY_NAME=' /etc/os-release
  id
} | tee ~/curso-linux/evidencias/sistema.txt

sudo apt update
sudo apt install -y tree
tree ~/curso-linux
```

SALIDA REPRESENTATIVA

```
/home/ubuntu/curso-linux
├── evidencias
│   └── sistema.txt
```

- `{ ...; }` agrupa comandos en el shell actual.
- `$(...)` sustituye un comando por su salida.
- `tee` muestra y guarda el reporte.
- `-y` confirma la instalación; úsalo sólo si ya revisaste el paquete.

Comprueba el resultado:

```
test -s ~/curso-linux/evidencias/sistema.txt \
&& echo "Evidencia creada correctamente"
```


Errores frecuentes

[Índice ↑](#)

- Confundir Ubuntu con el kernel Linux.
- Trabajar siempre como `root` en vez de usar `sudo` por comando.
- Ejecutar `apt upgrade` sin revisar qué cambiará.
- Usar `usermod -G` sin `-a` y perder grupos secundarios.

Reto 1 — Inventario reproducible

[Índice ↑](#)[Ver respuesta](#)

Crea `~/curso-linux/evidencias/inventario.txt` con hostname, distribución, kernel, usuario, grupos y ruta del ejecutable `bash`. No escribas esos valores a mano.

Criterios de comprobación

[Índice ↑](#)

- El archivo contiene seis datos con etiquetas legibles.
- Los valores provienen de comandos.
- El comando puede ejecutarse nuevamente sin producir errores.

Checklist

[Índice ↑](#)

- ☐ Distingo kernel y distribución.
- ☐ Distingo terminal, shell y escritorio.
- ☐ Sé consultar `man` y `--help`.
- ☐ Puedo buscar e instalar un paquete.
- ☐ Puedo interpretar UID, GID y grupos.

Estructura, enlaces y permisos

Tipos de archivo, rutas, jerarquía, montajes y el modelo de acceso para propietario, grupo y otros.

[Capítulo 3](#)[FHS](#)[chmod](#)[chown](#)

Objetivos

[Índice ↑](#)

- Reconocer tipos de archivos y rutas.
- Navegar la jerarquía principal de Linux.
- Crear enlaces y explicar su diferencia.
- inspeccionar montajes sin modificar discos.
- aplicar permisos y propietarios de forma consciente.

Antes de empezar

[Índice ↑](#)

Sitúate en la raíz del curso y crea el espacio de trabajo:



TERMINAL · BASH

```
cd ~/linux-desde-cero  
mkdir -p laboratorio/enlaces
```

En los ejemplos usaremos estos nombres concretos:

Nombre	Qué representa
<code>config.ini</code>	archivo original
<code>config-duro.ini</code>	enlace duro al original
<code>config-actual.ini</code>	enlace simbólico al original
<code>deploy.sh</code>	archivo para practicar permisos

3.1 Tipos de archivo

[Índice ↑](#)

El primer carácter de `ls -l` indica el tipo:

Carácter	Tipo	Ejemplo
-	archivo regular	texto, CSV, binario
d	directorio	<code>/home</code>
l	enlace simbólico	acceso alternativo
b / c	dispositivo de bloque/carácter	discos, terminales
s	socket	comunicación local
p	pipe con nombre	comunicación entre procesos

TERMINAL · BASH

```
file /etc/passwd /bin/ls /dev/null
ls -ld /etc/passwd /tmp /dev/null
```

`file` inspecciona el contenido o metadatos; la extensión no decide el tipo en Linux.

3.2 Enlaces

[Índice ↑](#)

SINTAXIS GENERAL



TERMINAL · BASH

```
ln <archivo_existente> <nuevo_enlace_duro>
ln -s <ruta_objetivo> <nuevo_enlace_simbolico>
```

Ejemplo resuelto: `config.ini` será el archivo existente y los otros dos nombres serán los enlaces.



TERMINAL · BASH

```
mkdir -p laboratorio/enlaces
printf 'version=1\n' > laboratorio/enlaces/config.ini
ln laboratorio/enlaces/config.ini laboratorio/enlaces/config-duro.ini
ln -s config.ini laboratorio/enlaces/config-actual.ini
ls -li laboratorio/enlaces
```

- `ln origen destino`: enlace duro al mismo inode.
- `ln -s objetivo enlace`: enlace simbólico que guarda una ruta.
- `ls -i`: muestra el inode; los enlaces duros comparten número.
- Un enlace simbólico puede cruzar sistemas de archivos, pero puede quedar roto.

3.3 Rutas o paths

[Índice ↑](#)

- Absoluta: comienza en `/`, por ejemplo `/var/log`.
- Relativa: comienza en el directorio actual, por ejemplo `../data`.
- `.`: directorio actual.
- `..`: directorio padre.
- `~`: home del usuario.



TERMINAL · BASH

```
pwd
realpath laboratorio/enlaces/config.ini
basename /var/log/syslog
dirname /var/log/syslog
```

3.4 Jerarquía principal

[Índice ↑](#)

Ruta	Propósito práctico
/home	datos y configuración de usuarios
/etc	configuración del sistema y servicios
/var	datos variables, cachés y logs
/tmp	temporales; no asumir persistencia
/usr	programas, bibliotecas y datos compartidos
/dev	dispositivos
/proc	vista del kernel y procesos
/run	estado desde el arranque



TERMINAL · BASH

```
ls -lah /\ncat /proc/meminfo | head
```

3.5 Acceso a sistemas de archivos

[Índice ↑](#)

En este curso se inspeccionan montajes; no se formatean discos.



TERMINAL · BASH

```
lsblk -f
findmnt /
df -hT /
```

SALIDA REPRESENTATIVA



SALIDA REPRESENTATIVA

TARGET	SOURCE	FSTYPE	OPTIONS
/	/dev/...	ext4	rw,relatime

- `lsblk -f`: dispositivos, sistema de archivos, etiquetas y montajes.
- `findmnt`: relación entre destino y origen.
- `df -hT`: espacio disponible y tipo; `-h` usa unidades legibles y `-T` agrega el tipo.
- ext4 es común en Ubuntu; XFS y Btrfs ofrecen decisiones diferentes, pero el usuario trabaja con la misma jerarquía.

3.6 Permisos

Índice ↑

SALIDA REPRESENTATIVA

```

-rwxr-x--- 1 alumno devops 1200 jul 4 10:00 deploy.sh
├──┬──┬──
│   │   │
│   │   │ dueño grupo
│   │   └──
│   │       otros: ---
│   └──┬──
│       │ grupo: r-x
│       └──┬──
│           │ dueño: rwx
│           └──┬──
│               │ archivo regular

```

Permiso	Archivo	Directorio	Valor
r	leer contenido	listar nombres	4
w	modificar	crear/eliminar entradas	2
x	ejecutar	atravesar/acceder	1

TERMINAL · BASH

```
touch laboratorio/enlaces/deploy.sh
chmod u=rwx,g=rx,o= laboratorio/enlaces/deploy.sh
chmod 750 laboratorio/enlaces/deploy.sh
sudo chown "$USER": "$(id -gn)" laboratorio/enlaces/deploy.sh
ls -l laboratorio/enlaces/deploy.sh
```

En `chown`, `$USER` se sustituye automáticamente por tu usuario actual y `$(id -gn)` por tu grupo principal. Por ejemplo, si ambos se llaman `ubuntu`, el comando efectivo equivale a:

TERMINAL · BASH

```
sudo chown ubuntu:ubuntu laboratorio/enlaces/deploy.sh
```

No copies `ubuntu:ubuntu` si tu cuenta tiene otro nombre; la versión con `$USER` se adapta sola.

- `chmod` : cambia bits de permiso.
- `chown usuario:grupo` : cambia dueño y grupo.
- `chgrp` : cambia sólo el grupo.
- `750` equivale a `rxr-x---`.

Práctica guiada resuelta

[Índice ↑](#)

La práctica crea este árbol, por lo que no necesitas preparar los archivos a mano:

SALIDA REPRESENTATIVA

```
laboratorio/proyecto/  
├─ bin/status.sh  
├─ config/app.env  
└─ logs/
```

TERMINAL · BASH

```
mkdir -p laboratorio/proyecto/{config,logs,bin}  
printf 'PORT=8080\n' > laboratorio/proyecto/config/app.env  
printf '#!/usr/bin/env bash\nnecho "servicio listo"\n' > laboratorio/proyecto/bin/status.sh  
chmod 640 laboratorio/proyecto/config/app.env  
chmod 750 laboratorio/proyecto/bin/status.sh  
ln -s ../config/app.env laboratorio/proyecto/bin/config-actual  
laboratorio/proyecto/bin/status.sh  
ls -l laboratorio/proyecto/config laboratorio/proyecto/bin
```

Salida principal:

SALIDA REPRESENTATIVA

```
servicio listo  
-rwxr-x--- ... status.sh  
lrwxrwxrwx ... config-actual -> ../config/app.env  
-rw-r----- ... app.env
```

El enlace muestra `rwx` por sí mismo; el acceso efectivo depende del objetivo y de los directorios de la ruta.

Si ves `Permission denied` al ejecutar `status.sh`, confirma primero `pwd` y luego ejecuta `ls -l laboratorio/proyecto/bin/status.sh`.

Errores frecuentes

[Índice ↑](#)

- Aplicar `chmod -R 777` para ocultar un problema.
- Confundir espacio de `df` con tamaño de archivos calculado por `du`.
- Crear un enlace simbólico relativo desde el directorio equivocado.
- Ejecutar `chown` o `rm` sin comprobar la ruta.

Reto 3 — Directorio compartido

[Índice ↑](#)[Ver respuesta](#)

Crea `laboratorio/compartido` para un equipo: dueño con control total, grupo con lectura/escritura/acceso y otros sin acceso. Dentro crea `app.env` como configuración no ejecutable, `check.sh` como script ejecutable y `check-actual` como enlace simbólico al script.

Criterios de comprobación

[Índice ↑](#)

- Los modos reflejan necesidades distintas para directorio, configuración y script.
- El script se ejecuta y la configuración no.
- `readlink` permite identificar el objetivo del enlace.

Checklist

[Índice ↑](#)

- ☐ Distingo tipo de archivo, extensión e inode.
- ☐ Uso rutas absolutas y relativas.
- ☐ Puedo explicar enlaces duros y simbólicos.
- ☐ Inspecciono montajes sin modificar discos.
- ☐ Interpreto y cambio permisos simbólicos y octales.

03

TRABAJO EN TERMINAL

El shell: comandos esenciales

Resolución de comandos, navegación y operaciones seguras con archivos hasta permisos y propiedad.

Capítulo 7.1–7.15

bash

cp · mv · rm

file · stat

Objetivos

[Índice ↑](#)

- Navegar y administrar archivos desde Bash.
- Consultar ayuda y verificar el efecto de cada comando.
- Buscar, medir, empaquetar y recuperar información.
- Aplicar opciones de seguridad antes de borrar o sobrescribir.

Antes de empezar

[Índice ↑](#)

Este capítulo forma una secuencia: cada apartado reutiliza archivos del anterior. Empieza desde la raíz del curso y prepara una copia limpia de los datos:

```
cd ~/linux-desde-cero
bash ejercicios-bash-scripting/preparar-lab.sh
mkdir -p laboratorio/shell/{entrada,salida,backup}
```

TERMINAL · BASH

Trabajaremos con `modelo.txt` como archivo original, `backup-modelo.txt` como copia y `modelo-validado.txt` como nombre nuevo. No uses archivos personales para estas prácticas.

7.1–7.3 Shell, comandos y directorio personal

[Índice ↑](#)

Bash interpreta palabras, expansiones, redirecciones y operadores. Un comando puede ser **builtin del shell** o un **ejecutable externo**:

Tipo	Descripción	Ejemplos
Builtin	Integrado en el propio shell; no existe como archivo en el sistema	<code>cd</code> , <code>echo</code> , <code>export</code> , <code>source</code>
Alias	Atajo definido en la sesión o en <code>.bashrc</code> ; envuelve otro comando	<code>ls</code> → <code>ls --color=auto</code> , <code>grep</code> → <code>grep --color=auto</code>
Función	Función definida en el shell o en archivos de configuración	funciones personalizadas en <code>.bashrc</code>
Externo	Archivo binario en el sistema de archivos; el shell lo localiza vía <code>\$PATH</code>	<code>ls</code> , <code>grep</code> , <code>awk</code> , <code>sed</code> , <code>find</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>cat</code> , <code>git</code> , <code>python3</code>

El shell resuelve en este orden: alias → función → builtin → externo.

¿Por qué importa? Los builtins son más rápidos (no crean un proceso nuevo) y pueden modificar el estado del shell actual. `cd`, por ejemplo, solo funciona como builtin: si fuera externo, cambiaría el directorio de un proceso hijo y el shell padre nunca lo notaría.

Usa `type` para identificar la naturaleza de cualquier comando:

```

TERMINAL · BASH

type cd
type ls
type echo

```

La salida depende de tu configuración; ejemplos comunes:

```

SALIDA REPRESENTATIVA

cd is a shell builtin
ls is aliased to `ls --color=auto`
echo is a shell builtin

```

Para saltar alias y llegar al ejecutable real, usa `type -a` o `which`:

TERMINAL · BASH

```
type -a ls      # alias → /usr/bin/ls → /bin/ls  (alias + externo)
type -a grep    # alias → /usr/bin/grep          (alias + externo)
type -a echo    # builtin → /usr/bin/echo        (builtin Y también externo)
type -a mkdir   # solo /usr/bin/mkdir           (puro externo, sin alias ni builtin)
type -a cd      # cd is a shell builtin         (solo builtin, sin archivo)

which ls        # /usr/bin/ls  (ignora el alias, devuelve el binario)
which grep      # /usr/bin/grep (ignora el alias, devuelve el binario)
which echo      # /usr/bin/echo (ignora el builtin, devuelve el binario externo)
which mkdir     # /usr/bin/mkdir (externo puro, sin alias ni builtin)
which cd        # (sin salida – cd es builtin, no tiene archivo ejecutable)
```

Directorio personal y navegación básica:

TERMINAL · BASH

```
echo "$HOME"
cd
pwd
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
/home/ubuntu
/home/ubuntu
```

7.4 Listar: `ls`

[Índice ↑](#)

TERMINAL · BASH

```
ls -lah
ls -lt laboratorio | head
```

- `-l` : formato largo.
- `-a` : incluye nombres que comienzan con `.`.
- `-h` : tamaños legibles junto con `-l`.
- `-t` : ordena por modificación.

No analices `ls` en scripts complejos: nombres con espacios o saltos de línea requieren herramientas como `find`.

7.5–7.8 Directorios y navegación

[Índice ↑](#)

TERMINAL · BASH

```
mkdir -p laboratorio/shell/{entrada,salida,backup}  
cd laboratorio/shell/entrada  
pwd  
cd -  
rmdir laboratorio/shell/backup
```

Después de `cd laboratorio/shell/entrada`, `pwd` debe terminar en `/laboratorio/shell/entrada`. `cd -` vuelve a la raíz del curso desde la que comenzaste. `rmdir` elimina `backup` porque todavía está vacío; más adelante los respaldos se crearán con otros nombres.

- `mkdir -p`: crea padres y tolera rutas existentes.
- `rmdir`: elimina sólo directorios vacíos.
- `cd -`: regresa a la ruta anterior.
- `pwd`: muestra la ruta efectiva actual.

7.9 Unidades y montajes

[Índice ↑](#)

Linux presenta los sistemas de archivos dentro de una sola jerarquía:



TERMINAL · BASH

```
lsblk -f  
findmnt /
```

No accedas a “una unidad” por letra; identifica su punto de montaje.

7.10 Copiar: `cp`

[Índice ↑](#)

SINTAXIS GENERAL



TERMINAL · BASH

```
cp [opciones] <origen> <destino>
```

En el ejemplo resuelto, `modelo.txt` es el origen. El primer `printf` crea ese archivo; no necesitas crearlo antes.



TERMINAL · BASH

```
printf 'accuracy=0.93\n' > laboratorio/shell/entrada/modelo.txt
cp -v laboratorio/shell/entrada/modelo.txt laboratorio/shell/backup-modelo.txt
cp -a laboratorio/shell/entrada laboratorio/shell/entrada-copia
```

- `-v`: informa operaciones.
- `-a`: copia recursivamente preservando metadatos apropiados.
- Para evitar sobrescritura accidental puede usarse `cp -i`.

7.11 Mover y renombrar: `mv`

[Índice ↑](#)

Sintaxis general: `mv <origen> <destino>`. Aquí renombraremos la copia `backup-modelo.txt` como `modelo-validado.txt`; el archivo original `entrada/modelo.txt` permanece intacto.



TERMINAL · BASH

```
mv -i laboratorio/shell/backup-modelo.txt laboratorio/shell/modelo-validado.txt
```

Dentro del mismo sistema de archivos, renombrar suele ser inmediato. Entre sistemas puede implicar copiar y eliminar.

7.12 Enlaces: ln

[Índice ↑](#)

El objetivo se escribe como `entrada/modelo.txt` porque el enlace estará dentro de `laboratorio/shell`. Esa ruta se interpreta desde la ubicación del enlace, no desde tu terminal.



TERMINAL · BASH

```
ln -s entrada/modelo.txt laboratorio/shell/modelo-actual
readlink laboratorio/shell/modelo-actual
```

`-s` crea un enlace simbólico; `readlink` muestra su objetivo almacenado.

7.13 Borrar: `rm`

[Índice ↑](#)

TERMINAL · BASH

```
rm -i laboratorio/shell/modelo-validado.txt
rm -rI laboratorio/shell/entrada-copia
```

- `-i` : pregunta por cada archivo.
- `-r` : recorre directorios.
- `-I` : una confirmación para una eliminación recursiva o numerosa.

Antes de un `rm -r`, ejecuta `ls` sobre la misma ruta. No uses `-f` como solución automática.

Ambos comandos pedirán confirmación. Responde `y` sólo después de comprobar que las rutas comienzan con `laboratorio/shell/`.

7.14 Identificar: `file` y `stat`

[Índice ↑](#)

TERMINAL · BASH

```
file laboratorio/shell/entrada/modelo.txt
stat -c '%F | %s bytes | %a | %n' laboratorio/shell/entrada/modelo.txt
```

SALIDA REPRESENTATIVA



SALIDA REPRESENTATIVA

```
ASCII text
regular file | 14 bytes | 644 | laboratorio/shell/entrada/modelo.txt
```

- `%F`: tipo.
- `%s`: bytes.
- `%a`: modo octal.
- `%n`: nombre.

7.15 Permisos y propiedad

[Índice ↑](#)

TERMINAL · BASH

```
chmod 640 laboratorio/shell/entrada/modelo.txt
chgrp "$(id -gn)" laboratorio/shell/entrada/modelo.txt
sudo chown "$USER":"$(id -gn)" laboratorio/shell/entrada/modelo.txt
```

- `chmod` : modo.
- `chgrp` : grupo.
- `chown` : dueño y opcionalmente grupo.

`$USER` y `$(id -gn)` se resuelven automáticamente. Si el usuario y grupo son `ubuntu`, el último comando equivale a `sudo chown ubuntu:ubuntu ...`.



EVIDENCIA Y CIERRE

Clase 1 completada

Al terminar la Clase 1, conserva una evidencia reproducible de tu sistema y de la estructura creada.



Identifico distribución, kernel, usuario, grupos y shell activo.



Navego y opero archivos con rutas, origen y destino claros.



Interpreto enlaces, propietario, grupo y permisos antes de modificarlos.

SIGUIENTE: CLASE 2 • SHELL ÚTIL Y ESCRITORIOS